



Asset Pulse — Windows executable distribution

This document explains how to **run** Asset Pulse on Windows from a single AssetPulseLauncher .exe, and how to **build** that executable from source.

The launcher does **not** bundle Docker. Asset Pulse runs as a Docker Compose stack (PostgreSQL 17 + FastAPI backend + Nginx-served React frontend). The .exe is a thin convenience wrapper that:

1. Confirms Docker Desktop is installed and running.
2. Bootstraps .env (with a randomly generated POSTGRES_PASSWORD) if it is

missing.

3. Runs `docker compose up -d --build` from the distribution folder.
4. Polls `/api/health` until the backend is ready.
5. Opens `http://localhost:8080` in your default browser.

1. Prerequisites (end-user machine)

- **Windows 10/11 64-bit.**
- **Docker Desktop** for Windows, installed and running. Download:

<<https://www.docker.com/products/docker-desktop/>>

- WSL 2 backend is the easiest. Docker Desktop's installer enables it for

you.

- First launch of Docker Desktop can take 1–2 minutes — wait for the whale

icon in the tray to stop animating before starting Asset Pulse.

- **~2 GB free disk** for the container images plus the database volume.
- **Ports 8080 and 8000** must be free on the host.

The launcher itself has no other dependencies — it is a single .exe.

2. Running Asset Pulse from the distribution

The distribution ZIP unpacks to a folder containing `docker-compose.yml`, `backend/`, `frontend/`, `db/`, `launcher/`, etc.

1. Right-click the ZIP → **Extract All...** to a folder you control

(e.g. `C:\Users\<you>\AssetPulse`). Avoid extracting inside `Program Files` — Docker Desktop cannot bind-mount paths there with WSL 2 by default.

2. Open the extracted folder.
3. Double-click `AssetPulseLauncher.exe` (if included), or



launcher\AssetPulseLauncher.bat as the fallback.

4. A console window appears. On first run it builds the images (a few minutes); subsequent runs start in seconds.

5. The default browser opens at [<http://localhost:8080>](http://localhost:8080).

Stopping the stack

From the same folder, in a terminal:

```
docker compose down          # stop containers, KEEP database
docker compose down -v       # stop AND DELETE the database volume (destroys data)
```

Closing the launcher console window does **not** stop the stack — containers continue to run in the background, which is usually what you want.

Viewing logs / status

```
docker compose ps
docker compose logs -f backend
docker compose logs -f frontend
docker compose logs -f db
```

Resetting credentials

The launcher only generates a .env if one is missing. To rotate the database password, edit .env in the project folder, then run:

```
docker compose down
docker compose up -d --build
```

3. Building AssetPulseLauncher.exe (developer machine)

The .exe must be produced on **Windows** — PyInstaller does not cross-compile PE binaries from Linux/macOS.

Prerequisites

- Python **3.10+** for Windows (python --version should work in PowerShell).
- Internet access (to install PyInstaller from PyPI).

Build

From a PowerShell prompt:

```
cd <repo-root>\launcher
.\build_windows_exe.ps1
```

The script installs PyInstaller, runs it in --onefile --console mode, and writes launcher\dist\AssetPulseLauncher.exe.

Copy that .exe to the project root (next to docker-compose.yml) before zipping the distribution. The launcher resolves docker-compose.yml relative to its own location, so the .exe must travel with the project.



4. What's inside the launcher

launcher/asset_pulse_launcher.py is the source of record. It is plain Python 3 and uses only the standard library, so it can also be run directly:

```
python launcher\asset_pulse_launcher.py
```

This is what AssetPulseLauncher.bat does — it is the documented fallback when shipping or running the .exe is not practical.

Behaviour summary:

Step	What it does	Failure mode
Locate root	Uses the .exe / script's parent folder	Aborts if docker-compose.yml is missing
Check Docker	docker --version, docker compose version, docker info	Prints install/start instructions and exits
Bootstrap .env	Copies .env.docker.example, generates a 24-char POSTGRES_PASSWORD	Warns if the example is missing
Start stack	docker compose up -d --build (live-streamed output)	Returns the compose exit code
Wait for health	Polls /api/health on 8080 then 8000, up to 240s	Warns and continues — user can investigate logs
Open browser	webbrowser.open("■@■")	Warns and prints the URL

The launcher never deletes containers, volumes, or images. It does not modify an existing .env. It does not phone home.

5. Troubleshooting

"Docker daemon is not running." — Start Docker Desktop, wait for the whale icon in the system tray to stop animating, then re-run the launcher.

"docker compose up exited with status 1" — Open a terminal in the project folder and run docker compose logs to see which service failed. Common causes: port 8080 or 8000 already in use, low disk space, or a stale image. docker compose pull then docker compose up -d --build usually clears it.

"Backend did not report healthy within the timeout." — The first build on a slow machine can exceed 4 minutes. Run docker compose logs -f backend and wait for Uvicorn running on Then visit [<http://localhost:8080>](http://localhost:8080).

SmartScreen warns about an unsigned executable. — Expected for a self-built binary. Click **More info** → **Run anyway**, or sign the .exe with your own code-signing certificate before distribution.

Antivirus quarantines the .exe. — Some AV tools heuristically flag PyInstaller bundles. Ship the .bat fallback alongside the .exe, or distribute the source and have users run python launcher\asset_pulse_launcher.py.